A Reproducible Arm-Based OpenAirInterface DU Testbed over Heterogeneous F1 Backhaul

Marc DUBOC and Ammar El Falou
King Abdullah University of Science and Technology (KAUST)
Thuwal, Saudi Arabia

***Abstract*—Multi-UAV fifth-generation (5G) cellular coverage requires a lightweight, repeatable cell building block replicated across the aerial fleet. We keep the 5G Core and central unit (CU) on the ground and define the repeated cell building block as an Arm-based OpenAirInterface (OAI) distributed unit (DU), radio, and F1 backhaul. Raspberry Pi 5 and Jetson Orin Nano DUs drive a Universal Software Radio Peripheral (USRP) B210 while F1 traverses Ethernet/fiber, Wi-Fi/Generic Routing Encapsulation (GRE), or 5G/WireGuard. Repeated end-to-end trials gate F1, registration, data, rollback, and a single-segment Public Warning System (PWS) alert concurrently displayed by commercial devices (phones and tablets) inside a Faraday cage. Twenty trials per fixed final configuration in the same enclosed lab retain per-run observations; mean downlink is 99.8 Mbit s^{-1} on tuned Ethernet, 76.7 Mbit s^{-1} over x86 cellular backhaul, and 68.4 Mbit s^{-1} on Jetson cellular backhaul; uplink ranges from 10.0–22.4 Mbit s^{-1}. Network-side PWS delivery latency ranges from 120–216 ms across bearers. Scheduler traces identify a Block Error Rate (BLER) adaptation mismatch as the dominant observed limiter; changing only the controller thresholds moves the dominant Modulation and Coding Scheme (MCS) from 5 to 24–27. A 106-PRB Raspberry Pi resource sample uses about 1.8 CPU cores and 1.4 GB RAM at 64.8 $^\circ C$ without late-packet or overflow markers. The measured core electronics weigh 0.657 kg (657.4 g). The testbed validates the repeated Arm DU/radio/backhaul unit that motivates future one-CU/many-UAV networks; flight and simultaneous multi-DU operation remain future work.**

***Index Terms*—5G NR, OpenAirInterface, CU/DU split, F1, AArch64, Jetson, SCTP, reproducibility, wireless backhaul, multi-UAV networks, aerial swarms**

I. Introduction

Aerial and rapidly deployable base stations can restore local service after infrastructure failures and at temporary events. The research problem is not only coverage: every airborne cell consumes payload mass, electrical power, and acquisition budget. A monolithic next-generation Node B (gNB) repeats the complete base-station compute stack on every aircraft, so fleet cost grows with the number of cells. Our objective is therefore to minimize the *incremental* equipment replicated per cell while retaining a modifiable, standards-based 5G New Radio (5G NR) path to a commercial terminal.

At fleet scale, the target is a repeatable per-cell unit, not merely a smaller one-off base station. The split removes CU-side RRC/PDCP/SDAP and x86 compute from every additional aircraft, leaving the Arm DU, radio, and backhaul as the replicated cell building block. One-DU validation cannot establish multi-cell capacity, but it can establish the exact end-to-end deployment contract that a fleet would repeat; that contract is the object evaluated here.

F1 is the appropriate functional boundary for that objective. Third Generation Partnership Project (3GPP) specifications allow one gNB central unit (gNB-CU) to serve multiple gNB distributed units (gNB-DUs). The F1 control plane (F1-C) carries the F1 Application Protocol (F1AP) over the Stream Control Transmission Protocol (SCTP); the F1 user plane (F1-U) carries the user-plane General Packet Radio Service Tunnelling Protocol (GTP-U) over the User Datagram Protocol (UDP) [1], [2]. This split keeps the latency-sensitive Medium Access Control (MAC), Radio Link Control (RLC), and physical layer (PHY) beside the radio while sharing higher layers and the 5G Core (5GC). Unlike a lower-layer fronthaul, F1 does not require a purpose-built radio unit or tight transport timing. It therefore combines per-cell airborne processing with an Internet Protocol (IP) backhaul that can traverse heterogeneous links.

We use OAI rather than a proprietary stack because its publicly inspectable 5G implementation already supports the B210 real-radio path and the 3GPP CU/DU split, and because source access is essential to complete the warning path across F1 and instrument scheduler decisions. OAI also preserves an existing x86 rollback baseline, so the Arm and transport experiments change one dimension at a time. Replacing the stack would require rebuilding those baselines and the warning path before addressing the airborne question. OAI deployments have conventionally used general-purpose x86 systems [3]; moving only the DU to lower-mass 64-bit Arm compute tests whether the F1 boundary produces a practical payload benefit.

Three implementation problems still dominate: an embedded host must sustain real-time radio processing rather than merely compile OAI; embedded Linux distributions can omit SCTP, which is mandatory for F1-C; and a working F1 association can coexist with poor radio scheduling, wrong-interface traffic, or no usable handset service.

We address these problems with an evidence-driven OAI testbed. A ground 5GC/CU is paired with either a Raspberry Pi 5 or Jetson Orin Nano DU and a B210. The Pi provides the lowest-cost, lowest-mass entry point; the Jetson provides more compute headroom and matches the target embedded platform. Ethernet is evaluated as the deterministic wired baseline: while tested over copper in the laboratory, the same packetized F1 transport maps directly to optical fiber for tethered UAV deployments and long fixed segments before

transitioning to untethered wireless backhaul. Wi-Fi/GRE creates an observable wireless route, while a Quectel 5G modem with WireGuard provides stable, protected F1 endpoints over a mobile packet session. The workflow separately gates F1 packet placement, radio-frequency (RF) readiness, PWS reception, registration, Protocol Data Unit (PDU) session, Internet reachability, throughput, and rollback.

Previous work implemented SIB8 warning transmission in a modified monolithic OAI gNB and identified CU/DU support as future work [4]. This paper makes the tested request path functional across F1. Its contributions are:

1) a tested single-segment OAI PWS request path across F1 in which the CU issues F1AP warning control, the DU schedules SIB8, and commercial devices (phones and tablets) decode and display the broadcast alert, evaluated with safe input values, the pinned OAI revision, and the released patch;
2) real-radio OAI 5G split-DU operation on both Raspberry Pi 5 and Jetson Orin Nano, including native AArch64 compilation and a tested, rollback-safe SCTP-enabled Jetson kernel;
3) an instrumented diagnosis of an OAI Modulation and Coding Scheme (MCS) collapse: scheduler traces identify a Block Error Rate (BLER) controller mismatch as the dominant observed limiter and show recovery after BLER retuning; and
4) a fleet-oriented testbed contract evaluated across x86, two Arm hosts, and three F1 bearers: pinned software, path and handset gates, rollback, resource observations, and payload sizing define the COTS unit replicated behind a shared CU.

All 20 repetitions per setup used the same enclosed RF environment, access profile, handset, OAI pin, and workflow. Host, bearer, and named runtime or BLER changes distinguish setups. All 400 DL and UL observations are retained; Section V reports means and dispersion as configuration feasibility, not intrinsic bearer speed. We validate one DU at a time: simultaneous multi-DU CU capacity, inter-cell coordination, and swarm flight are enabled research directions, not results claimed by this paper. The multi-UAV use case is therefore a deployment requirement on the repeated unit, not a flight result.

II. Related Work and Positioning

A. From complete aerial cells to an incremental DU

Early open aerial radio access network (RAN) testbeds established feasibility by lifting most or all base-station computation. Flying Rebots placed an OAI Long-Term Evolution (LTE) evolved Node B (eNB) on commodity x86 compute aboard a custom airframe weighing roughly 2 kg before the communications payload [5]. SkyRAN carried an OAI LTE eNB and Evolved Packet Core (EPC) on separate Intel i7 and J1900 single-board computers, together with a B210 radio chain, on a heavy-lift industrial hexacopter [6]. SkyCell later paired an Intel NUC and B210 with an srsLTE eNB on the same aircraft class [7]. These designs tied the airframe to the weight, power, and cost of an x86 cellular site carried aloft.

The closest predecessor is Mundlamuri *et al.*, who already demonstrated an OAI 5G aerial DU, a B200mini, a Quectel 5G modem, wireless F1 backhaul, and approximately 30 Mbit s^{-1} to commercial terminals [8]. Accordingly, the novelty claimed here is neither the aerial DU concept nor wireless F1. That publication does not report the DU compute host, aircraft model, payload power or mass, Arm enablement, PWS delivery, or a cross-bearer benchmark. Our differentiation is a replication contract: the incremental DU is named, pinned, measured, exercised across heterogeneous paths, and paired with explicit service gates and rollback.

B. Arm execution and public-warning delivery

OAI is an open fourth-generation (4G)/5G implementation commonly evaluated on x86 commodity compute [3]. OAI subsequently added AArch64 compilation through the single instruction, multiple data (SIMD) portability layer SIMD Everywhere (SIMDe) [9], but source portability alone does not prove that an embedded board can sustain a real-time PHY and Universal Serial Bus (USB) radio. A 2024 6GSPACE report compiled OAI on a Raspberry Pi 4 and exercised the RF simulator, while recording unresolved runtime and architecture limitations; it did not demonstrate a real-radio 5G split DU [10]. The literature reviewed does not document the same real-radio split-DU path on both Raspberry Pi 5 and Jetson Orin Nano. Our contribution includes the system work hidden by a nominally successful compile: bounded build memory, Arm Neon/SIMDe execution, USB and central processing unit (CPU) tuning, and, on Jetson, an SCTP-capable kernel with a rollback path.

PWS literature has exposed security weaknesses using a commercial-stack testbed [11]. More recent work added System Information Block 8 (SIB8) warning transmission to a modified monolithic OAI gNB, explicitly leaving disaggregated CU/DU support for future work [4]. The present work extends that monolithic implementation: F1AP standardizes the Write-Replace Warning procedure between CU and DU [2], but the OAI request path still had to be completed and validated. In our testbed the CU sends a single-segment warning over F1-C, the remote DU schedules SIB8, and multiple commercial devices (phones and tablets) concurrently display the alert, confirming cell-broadcast reception. The tested profile uses identifier `0x1112`, serial `0xFF00`, data-coding scheme `0x48`, and mode `0`. The released patch provides the complete CU–F1–DU–UE implementation on the pinned OAI revision. We do not claim the response, deduplication, cancel, or multi-segment portions of the complete procedure. This matters operationally because a PWS observation simultaneously exercises CU logic, F1 signalling, DU scheduling, RF transmission, and multi-device behavior.

C. Failure diagnosis and benchmark gap

OAI documents a BLER-driven controller that raises or lowers MCS around configured thresholds [12]. Prior aerial

TABLE I
MASS AND DOWNLINK POSITIONING AGAINST REPRESENTATIVE OPEN AERIAL CELLULAR TESTBEDS. HETEROGENEOUS BANDWIDTHS, RADIOS, TERMINALS, AND TRAFFIC PREVENT A CONTROLLED RANKING.

System	Airborne RAN and compute	Reported airborne mass	Reported downlink outcome	Position relative to this work
Flying Rebots [5]	OAI LTE eNB; x86 compute and USRP	2 kg airframe before payload	$\sim$6 Mbit s^{-1} relay vs. 4 Mbit s^{-1} direct	Complete LTE eNB aloft; no 5G F1 split, Arm DU, or PWS.
SkyRAN [6]	OAI LTE eNB/EPC; Intel i7/J1900; B210	Communications payload not reported	0.9–0.95$\times$ measured optimum	Complete access and core stack aloft; mass not itemized.
SkyCell [7]	srsLTE eNB; Intel NUC; B210	Communications payload not reported	Up to 35% throughput improvement	Mobility-focused LTE platform; no OAI 5G, F1, or PWS.
Mundlamuri *et al.* [8]	OAI 5G DU; host not reported; B200mini/modem	Not reported	30 Mbit s^{-1}	Closest aerial DU and wireless F1; no reported Arm DU or PWS.
This work	OAI 5G DU; Raspberry Pi 5 or Jetson; B210/modem	0.657 kg measured core	99.8 Mbit s^{-1} mean Ethernet; 76.7 Mbit s^{-1} x86 and 68.4 Mbit s^{-1} Jetson cellular ($n = 20$)	Itemized incremental-DU mass; flight pending.

demonstrations report a working configuration or a throughput point, but not the case encountered here: healthy F1/SCTP and UE attachment with downlink trapped at 12–22 Mbit s^{-1} because the filtered BLER remained outside the default adaptation window. The symptom appeared during the Ethernet experiment, but the adaptation mechanism is bearer-independent: it can occur whenever the radio-link BLER remains outside the configured window. Section V provides the observed BLER mechanism and the paired before/after intervention. The reported throughput remains an end-to-end result of the complete validated configuration rather than a universal scheduler optimum.

Unlike prior single-platform demonstrations, our public COTS matrix crosses x86, Raspberry Pi 5, and Jetson; monolithic and split execution; and three F1 bearers. Exact software, packet-path evidence, handset gates, rollback, and an itemized mass envelope make the prototype reproducible without claiming unmeasured flight endurance.

III. TESTBED AND REPRODUCIBILITY METHOD

A. Architecture

Fig. 1 shows the system. The ground host runs the OAI 5GC and CU. The CU terminates Radio Resource Control (RRC), Packet Data Convergence Protocol (PDCP), and Service Data Adaptation Protocol (SDAP); the selected Arm DU runs the MAC/RLC/PHY layers defined above and drives a B210 over USB 3. The access cell uses band n78, 30 kHz subcarrier spacing, and 106 Physical Resource Blocks (PRBs). Commercial devices serve as end-to-end UEs. PWS adds an observable application path: the CU issues an F1AP Write-Replace Warning request, the DU schedules SIB8, and commercial devices (phones and tablets) display the broadcast warning [2].

The figure shows the DU validated in this work as DU_1. From the CU's point of view, the same F1 architecture can fan out to $\mathrm{DU}_2 \ldots \mathrm{DU}_N$, each with its own F1 endpoints, access cell, and airborne payload. Replicating the green payload while retaining the ground 5GC/CU is the intended multi-UAV extension; the dashed branch is architectural scale-out, not a simultaneous experiment.

The B210 provides up to 56 MHz real-time bandwidth and a 61.44-mega-sample-per-second (61.44 MSps) quadrature path over SuperSpeed USB 3 [13]. The Quectel case uses two independent cells: a donor cell serves the modem, while the B210 access cell serves the handset. WireGuard gives stable F1 endpoint addresses over the modem's dynamic packet session. This separation prevents the common false positive in which the handset joins the donor cell while the experiment is attributed to the access DU.

B. Testbed Configuration and Security Boundary

The testbed configuration pins the OAI revision and applies modular patches, including the PWS patch at `patches/sib8/oai-pws-sib8-cu-du.patch`, with safe configuration templates [14]. Private runtime inputs, credentials, database identifiers, and keys are strictly excluded. All radio emissions are contained within the lab environment.

The access B210 and intended handset operate inside the lab Faraday cage on an isolated test public land mobile network (PLMN), using explicitly test-only warning text. A run is invalid if radio containment, the intended UE, the selected access cell, or exclusive lab ownership cannot be proven.

The operator workflow enforces the sequence below:

1) discover the selected host, B210, modem, addresses, routes, USB speed, OAI commit, and patch state;
2) acquire an exclusive operator lock, generate the scenario config, and launch the 5GC, CU, transport, and DU in dependency order;
3) prove core health, F1-C SCTP, F1-U GTP-U, RF synchronization, and the absence of F1 leakage onto management interfaces;
4) separately record handset PWS, registration, PDU session, external data network (DN) reachability, Internet access, downlink/uplink throughput, and timestamps;
5) stop all processes, check residue, and reproduce the Ethernet rollback.

This distinction matters: F1 setup and an RF-ready message are machine-side milestones, not evidence of handset service. A scenario is accepted only when the relevant phone-visible gates also pass.

IV. NATIVE AARCH64 AND JETSON ENABLEMENT

A. Building OAI on Arm

The working method is a native build on the target, not an x86 binary copied to the board. OAI's SIMDe-backed

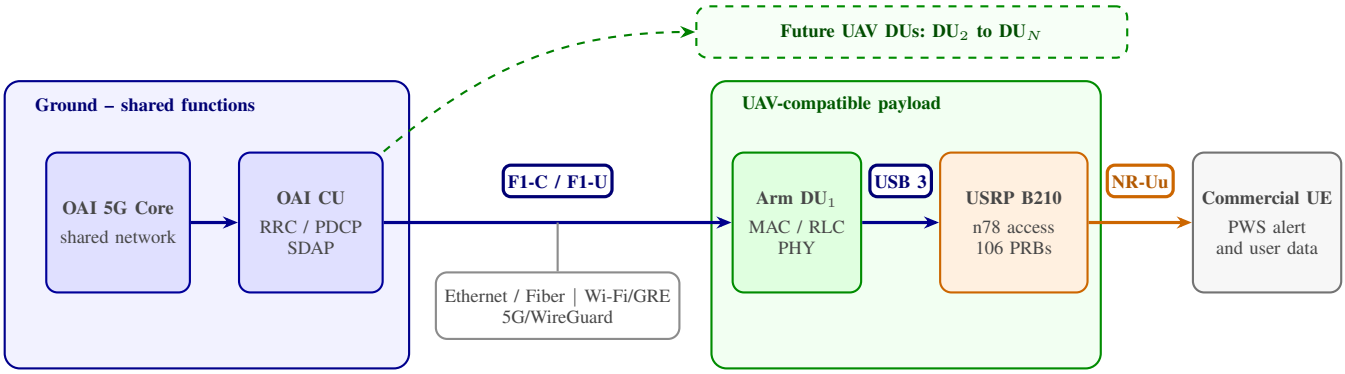

Fig. 1. Shared-CU testbed architecture. The solid DU_1 path is validated; the dashed $DU_2 \ldots DU_N$ branch shows the multi-UAV scale-out direction. Management traffic stays outside the selected F1 bearer, and packet captures verify the actual path.

code maps the x86-style vector application programming interface (API) used throughout the PHY onto AArch64/Neon-compatible implementations [9]. We freeze the source tree to commit `102965a6`, install dependencies on the Arm host via `./build_oai-I`, and build the required gNB/USRP binary using `./build_oai-wUSRP--ninja--gNB-C`.

Four details made this repeatable. The `libsctp-dev` and `lksctp-tools` packages are installed before the build so compile-time headers and runtime diagnostics agree. The USRP Hardware Driver (UHD) and its B210 images are installed locally; the USB radio cannot be treated as a remote x86 peripheral. Build parallelism is bounded and swap is available because an 8 GB board can otherwise fail during compilation or linking. Finally, the resulting `nr-softmodem`, shared parameter library, UHD backend, and exact source identity are verified on the target. No Jetson-specific OAI source fork is required for AArch64 itself; the lab's PWS and instrumentation changes remain explicit patches on the pinned tree.

B. Why the stock Jetson kernel failed

The Jetson Orin Nano ran NVIDIA Linux for Tegra (L4T) R36.4.4 / JetPack 6.2 with kernel `5.15.148-tegra`. The stock kernel configuration left `CONFIG_IP_SCTP` unset; consequently, `modprobesctp` failed and SCTP socket creation was rejected by the kernel. This blocks both F1-C signalling and pre-radio F1 validation before radio processing starts.

A standalone `sctp.ko` compiled from generic headers was not a safe fix: NVIDIA's Board Support Package (BSP) relies on out-of-tree GPU, display, audio, and Tegra drivers. If the running kernel and out-of-tree modules disagree on `CONFIG_LOCALVERSION` or compiler symbols, module verification fails, disabling GPU and display capabilities.

The reproducible kernel procedure follows these steps [15]:

1) inventory the stock L4T release and active boot entry;
2) obtain matching R36.4.4 public sources (in-tree kernel, NVIDIA out-of-tree, and display trees);
3) enable `CONFIG_IP_SCTP=m` and `CONFIG_INET_SCTP_DIAG=m` under an isolated local version;
4) compile natively with a 4 GB swap file and 4 parallel jobs, staging the image and coherent modules;
5) generate a matching initrd and register an alternate `extlinux.conf` entry to keep the stock kernel bootable.

After reboot, `checksctp`, a Python SCTP socket, and loopback `sctp_darn` all passed; GPU, display, and Ethernet drivers remained loaded. This validation is as important as the successful kernel build.

C. Real-time runtime profile

Kernel support alone did not make the Jetson a stable DU. At USB 2 speed (480 Mbit s^{-1}), the B210 produced continuous UHD receive overflows. A real SuperSpeed path at 5 Gbit s^{-1} (`5000M`) was required. The validated runtime sets maximum-performance mode, locks clocks, disables USB autosuspend, sets `usbfs_memory_mb=1000`, runs the DU on CPUs 1–5, and pins the active extensible Host Controller Interface (xHCI) interrupt to CPU 0. Selecting the active interrupt by increasing counters in `/proc/interrupts` removed a repeatability trap. Warning-level OAI logging reduces avoidable real-time pressure.

V. Benchmarks and MCS-Collapse Diagnosis

A. Measurement protocol and throughput benchmarks

Every supported scenario was evaluated in repeated end-to-end trials under identical experimental conditions. The Faraday cage, intended handset, n78 profile (30 kHz subcarrier spacing, 106 PRBs), pinned OAI commit, and verification gate sequence remained fixed; host compute and F1 transport bearer were the independent variables. Each trial required successful execution of all gates: F1 association, PWS broadcast, UE registration, PDU session establishment, local and external IP reachability, clean process termination, and baseline rollback. Packet captures verified that F1-C and F1-U traffic traversed only the intended bearer.

In total, 400 throughput observations were recorded (20 DL and 20 UL runs per configuration across 10 setups). Two-sided 95% confidence intervals (CIs) use Student's $t_{0.975,19} = 2.093$. Active F1 round-trip time (RTT) was sampled via ICMP during sustained traffic: Ethernet/fiber averaged $\sim$20 ms (11–30 ms),

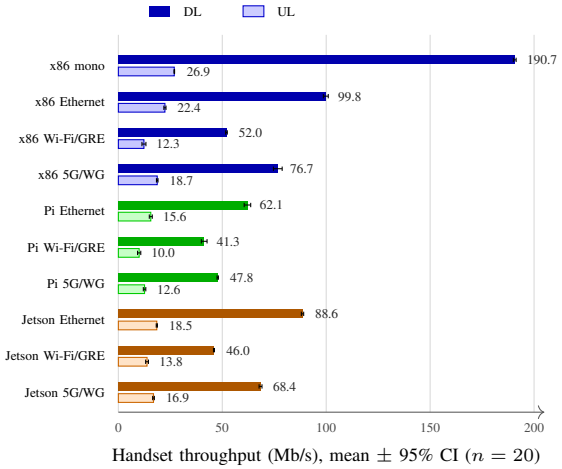

Fig. 2. Mean handset Downlink (DL, solid) and Uplink (UL, light) throughput across all 10 configurations ($n = 20$ per setup, 400 total observations).

Wi-Fi/GRE recorded ∼25 ms (15–40 ms) due to wireless contention, and 5G/WireGuard registered ∼40 ms (25–60 ms) due to donor-cell scheduling and tunnel encryption.

Fig. 2 summarizes all DL and UL results. The benchmark demonstrates clear consistency across platforms:

- **Host performance hierarchy:** On wired Ethernet, x86 leads with 99.8 Mbit s^{-1} DL / 22.4 Mbit s^{-1} UL; the Jetson Orin Nano reaches 88.6 Mbit s^{-1} DL / 18.5 Mbit s^{-1} UL (within 11% of x86), confirming that its 6-core Arm Cortex-A78AE architecture comfortably sustains disaggregated 5G NR baseband processing. The Raspberry Pi 5 reaches 62.1 Mbit s^{-1} DL / 15.6 Mbit s^{-1} UL, establishing a viable low-cost, ultra-lightweight entry point.
- **Bearer impact:** Across all hosts, Ethernet achieves the highest throughput and tightest dispersion, followed by 5G/WireGuard (e.g., Jetson delivers 68.4 Mbit s^{-1} DL / 16.9 Mbit s^{-1} UL). Wi-Fi/GRE throughput is bounded by uncontrolled RF contention on the shared campus network.
- **Uplink scaling:** Uplink throughput is proportional across all modes, constrained by the TDD slot format (∼20% UL allocation) and UE transmit limits.

B. Observed mechanism

During initial deployment, the Ethernet split configuration exhibited an unexpected throughput collapse, stalling below 23 Mbit s^{-1} with the downlink MCS locked to its lowest allowed value of 5. This occurred despite healthy F1-C signalling, stable SCTP associations, and robust radio channel conditions.

Tracing `get_mcs_from_bler()` in the OAI MAC scheduler revealed the underlying cause. The scheduler tracks an exponentially smoothed BLER:

$$\widehat{b}_k = 0.9\,\widehat{b}_{k-1} + 0.1\,b_k, \tag{1}$$

and updates MCS according to configured bounds: incrementing only when $\widehat{b}_k < b_{\text{lower}}$ and decrementing when $\widehat{b}_k > b_{\text{upper}}$. The default OAI source thresholds are $(b_{\text{lower}}, b_{\text{upper}}) = (0.05, 0.15)$. Under live split-RAN traffic, initial-round HARQ retransmissions naturally hovered between 22% and 35%, keeping $\widehat{b}_k$ above 0.18. Consequently, the controller could never trigger an MCS increment, continuously penalizing the link until it hit the MCS 5 floor.

TABLE II
BLER-Threshold Intervention and Performance Recovery

Metric	Before intervention	After retuning
DL target BLER window	0.05–0.15	0.25–0.35
Dominant scheduled MCS	5 (locked to floor)	24 and 27
Observed DL BLER	22–35%	∼22%
Handset DL throughput	up to 23 Mbit s^{-1}	99.8 Mbit s^{-1} ($n = 20$)
External-DN RTT	up to seconds (buffered)	11–30 ms

TABLE III
F1 Transport Latency and PWS Alert Delivery

F1 Bearer	F1 RTT	CU→DU	Handset Alert	
	Range (ms)	F1AP (ms)	Range (s)	Median (s)
Ethernet	11–30	120	0.3–0.9	0.8
Wi-Fi / GRE	15–40	182	0.4–1.4	1.0
5G / WireGuard	25–60	216	0.5–1.9	1.3

Mean stage pipeline budget: CU encode (2 ms) → F1-C transport (17 ms) → SI window wait (132 ms) → DU sched. (3 ms) → UE decode & display (970 ms).

C. BLER retuning and recovery

To resolve this limitation, we reconfigured the DU controller with relaxed thresholds: $b_{\text{DL,lower}} = 0.25$, $b_{\text{DL,upper}} = 0.35$, $b_{\text{UL,lower}} = 0.15$, and $b_{\text{UL,upper}} = 0.35$. All physical-layer parameters, radio attenuation, BWP definitions, and network transport remained strictly unchanged.

Table II shows the effect of this single-variable intervention: the scheduler immediately unlocked higher modulation schemes, with MCS 24 and 27 dominating active slots. Downlink throughput recovered from 23 Mbit s^{-1} to 99.8 Mbit s^{-1} ($n = 20$), confirming that the initial bottleneck was entirely an algorithmic mismatch between the controller's expectation and the real-radio HARQ dynamics.

D. PWS delivery across F1

Table III evaluates the disaggregated Public Warning System path. Network-side latency—measured from the CU `WRITE-REPLACE WARNING REQUEST` generation to DU SIB8 broadcast scheduling—ranged from 120 ms on Ethernet to 216 ms on 5G/WireGuard. As shown in the timing budget breakdown, CU F1AP encoding (2 ms) and DU scheduling (3 ms) introduce minimal overhead. The dominant network-side delay is the periodic System Information (SI) window alignment (132 ms mean).

End-to-end alert visibility on the commercial handset averaged 0.8–1.3 s across bearers. The majority of this latency is governed by handset-internal baseband decoding and Android/iOS notification rendering (970 ms mean), verifying that the disaggregated F1 backhaul adds negligible delay to emergency warning delivery.

VI. Embedded Resources and Payload Mass

Host-level resource measurements provide embedded execution evidence. A 51-PRB Raspberry Pi 5 sample recorded a

TABLE IV
MEASURED CORE MASS AND B200MINI BOARD-ONLY PROJECTION

Item	Mass (g)
Jetson Orin Nano board	151.4
USRP B210 with access antennas	217.3
Quectel modem kit with antennas	288.7
Measured core electronics	657.4
Regulator, harness, mounting	100.0
Estimated integration	757.4
B200mini board (vs. B210 assembly)	24.0
Projected mini-radio integration*	564.1

*Upper-bound mass reduction before adding mini-radio antennas and enclosure; the B200mini path has not undergone end-to-end validation.

CPU load of 2.04, 1.5 GB RAM used, 67.0 °C, no throttling, and no UHD underruns. At 106 PRBs, another sample used about 1.8 CPU cores and 1.4 GB RAM at 64.8 °C, with no late-packet or overflow markers after thread pinning and with PWS passing. These are synchronized samples, not cross-platform averages.

No aircraft has yet carried the testbed, so the mass model is anchored to the Jetson, B210, and Quectel configuration that passed attach, PWS, user-plane, and rollback gates. Table IV separates measured electronics from minimum integration allowances; the allowance yields a 0.757 kg integration estimate.

Ettus specifies a 24 g board-only mass for the B200mini, compared with the much larger B200/B210 form factor [13]. Substituting that board for the measured 217.3 g B210-and-antenna assembly gives $757.4 - 217.3 + 24.0 = 564.1$ g. This is deliberately reported as a projection: antennas, protection, mounting, and validation can reduce the 193.3 g upper-bound saving. It also explains why the validated B210 value, rather than the lighter untested radio, remains the headline.

With the engineering rule that payload mass occupy at most 70% of advertised capacity, the estimated integration requires at least $0.757/0.70 = 1.08\,\mathrm{kg}$ of payload rating. The mini-radio projection would start at 0.81 kg before its missing allowances. Vehicle selection still requires battery-input, thermal, vibration, center-of-gravity, electromagnetic-compatibility, and failsafe tests.

VII. CONCLUSION

Repeated end-to-end trials in one fixed enclosed lab validated single-segment PWS and data across Ethernet, Wi-Fi/GRE, and 5G/WireGuard F1 bearers. Jetson delivered mean 68.4 Mbit s^{-1} DL and 16.9 Mbit s^{-1} UL over 5G/WireGuard ($n = 20$); retuned Ethernet averaged 99.8 Mbit s^{-1} ($n = 20$). Network-side PWS latency ranged from 120 ms (Ethernet) to 216 ms (cellular). Changing only the BLER thresholds moved the dominant MCS from 5 to 24–27. Core electronics weigh 0.657 kg; the 0.757 kg integration estimate requires a 1.08 kg payload rating. The Arm DU/radio/backhaul payload, pinned software, path and handset gates, and rollback form the replicated cell building block. Keeping CU-side functions on the ground makes this bounded unit—not a complete x86 base station—what fleet size multiplies. Measured payload power, concurrent DUs, and controlled flight remain future work.

REFERENCES

[1] 3GPP, "TS 38.401: NG-RAN; architecture description," 3rd Generation Partnership Project, Technical Specification, 2025, release 18, version 18.6.0. [Online]. Available: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3219

[2] ——, "TS 38.473: NG-RAN; F1 application protocol (F1AP)," 3rd Generation Partnership Project, Technical Specification, 2026, release 18, version 18.10.0. [Online]. Available: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3260

[3] F. Kaltenberger, A. P. Silva, A. Gosain, L. Wang, and T.-T. Nguyen, "OpenAirInterface: Democratizing innovation in the 5G era," *Computer Networks*, vol. 176, p. 107284, 2020.

[4] A. Abou Hasna, N. Chendeb, and A. El Falou, "From spoofing to trust: Emergency alerts spoofing testbed and cross-cell verification," 2026. [Online]. Available: https://arxiv.org/abs/2604.24404

[5] R. Gangula, O. Esrafilian, D. Gesbert, C. Roux, F. Kaltenberger, and R. Knopp, "Flying rebots: First results on an autonomous UAV-based LTE relay using OpenAirInterface," in *Proc. IEEE International Workshop on Signal Processing Advances in Wireless Communications (SPAWC)*, 2018, pp. 1–5.

[6] A. Chakraborty, E. Chai, K. Sundaresan, A. Khojastepour, and S. Rangarajan, "SkyRAN: A self-organizing LTE RAN in the sky," in *Proc. ACM Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, 2018, pp. 280–292.

[7] L. Ferranti, L. Bonati, S. D'Oro, and T. Melodia, "SkyCell: A prototyping platform for 5G aerial base stations," in *Proc. IEEE WoWMoM*, 2020, pp. 329–334.

[8] R. Mundlamuri, O. Esrafilian, R. Gangula, R. Kharade, C. Roux, F. Kaltenberger, R. Knopp, and D. Gesbert, "Integrated access and backhaul in 5G with aerial distributed unit using OpenAirInterface," in *Proc. 17th ACM Workshop on Wireless Network Testbeds, Experimental Evaluation and Characterization (WiNTECH)*, 2023, demo. [Online]. Available: https://www.eurecom.fr/publication/7304

[9] OpenAirInterface Software Alliance, "OpenAirInterface5G v2.1.0 release notes: ARM compilation through SIMDe," https://gitlab.eurecom.fr/oai/openairinterface5g/-/tags/v2.1.0, 2024, accessed: 2026-07-20.

[10] SnT/SIGCOM 6GSPACE Lab, "Past and current activities on 5G and 6G NTNs using OAI at the 6GSPACE lab of SnT/SIGCOM," EURECOM/OAI NTN Meeting presentation, 2024, 5 April 2024; accessed: 2026-07-20. [Online]. Available: https://openairinterface.org/wp-content/uploads/2024/03/2024-04-05_SnT_EURECOM-OAI-NTN-Meeting.pdf

[11] E. Bitsikas and C. Pöpper, "You have been warned: Abusing 5G's warning and emergency systems," in *Proc. 38th Annual Computer Security Applications Conference (ACSAC)*, 2022, pp. 561–575.

[12] OpenAirInterface Software Alliance, "OpenAirInterface 5G MAC usage and scheduler documentation," Project documentation, accessed: 2026-07-20. [Online]. Available: https://github.com/OPENAIRINTERFACE/openairinterface5g/blob/develop/doc/MAC/mac-usage.md

[13] Ettus Research, "B200/B210/B200mini/B205mini/B206mini hardware and physical specifications," https://kb.ettus.com/B200/B210/B200mini/B205mini/B206mini, accessed: 2026-07-20.

[14] OAI CU/DU Lab Maintainers, "OAI CU/DU Lab: Reproducible 5G Split-RAN artifact," https://github.com/promaaa/oai-cu-du-lab, 2026, accessed: 2026-08-13; OAI pin, feature patches, and operator tooling.

[15] promaaa, "Jetson Orin Nano custom SCTP kernel and OOT drivers guide," https://github.com/promaaa/jetson-kernel-sctp, 2026, accessed: 2026-07-20.